



Facultatea
de Automatică
și Calculatoare
Universitatea Tehnică
din Cluj-Napoca



Fundamental Programming Techniques

Assignment 3 - Support Presentation (Part 2)

Computer Science Department, TUCN

Spring 2026

Reflection Techniques Example #1

- **Problem:** Given an instance of any object, extract the fields and the corresponding values
- Line of code 3 - obtain the class of the object and request the declared fields of that class
- For each declared field:
 - ▶ Line 4 - Set the field accessible because most of them are private => need to change permissions
 - ▶ Line 7 - Return the value of the current field contained in the specified object
 - ▶ Line 8 - Print the field's name and its value

```
1 public class ReflectionExample {
2     public static void retrieveProperties(Object object) {
3         for (Field field: object.getClass().getDeclaredFields())
4             {
5                 field.setAccessible(true);
6                 Object value;
7                 try {
8                     value = field.get(object);
9                     System.out.println(field.getName() + "=" + value);
10                } catch (IllegalArgumentException e) {
11                    e.printStackTrace();
12                } catch (IllegalAccessException e) {
13                    e.printStackTrace();
14                }
15            }
16    }
```

Take a look on this implementation:
https://gitlab.com/utcn_dsrl/pt-reflection-example

Reflection Techniques Example #2

- **Problem:** Propose a smart solution for constructing the DAO classes using Generics and Reflection
- Line 1 - Define a class *AbstractDAO* that defines the common operations for accessing a table: *insert*, *update*, *delete*, *findByld*, *findAll*. Define the operations on the specified generic type **<T>**. T can be any Java Model Class that is mapped to the Database, and has the same name as the table and the same instance variables and data types as the table fields.
- Line 4, Line 8 - For each *AbstractDAO* object obtain the class of the generic type T

```
1 public class AbstractDAO<T> {  
2     protected static final Logger LOGGER = Logger.getLogger(AbstractDAO.class.getName());  
3  
4     private final Class<T> type;  
5  
6     @SuppressWarnings("unchecked")  
7     public AbstractDAO() {  
8         this.type = (Class<T>) ((ParameterizedType) getClass().getGenericSuperClass()).getActualTypeArguments()[0];  
9     }  
10 }
```

Take a look on this implementation: https://gitlab.com/utcn_dsrl/pt-reflection-example

Reflection Examples #3

- **Problem:** Propose a smart solution for constructing the DAO classes using Generics and Reflection - findById method.
- Line 14 - A generic query is build using the id field name. The value *id* can be replaced in a generic way by defining a custom annotation for PK and obtaining the name of the annotated field.
- Line 1-8 - Build the generic select query using the name of the class previously obtained. The approach considers that the name of the Java model class is the same with the table db. For a more generic approach, define a custom annotation for the table name, where the developer can specify a name different from the name of the class.
- Line 16 - the connection is obtained from the ConnectionFactory previously defined.
- Line 18 - The *id* is considered to be of type int. For a more generic approach, the type of the PK can be specified in a generic way, similarly with the generic Type T.
- line 21 - Using the *ResultSet* obtained after executing the statement, we will obtain the list of objects. Only the first element is of interest, since the db should contain only one entry with the specified id.

```
1 private String createSelectQuery(String field) {
2     StringBuilder sb = new StringBuilder();
3     sb.append("SELECT ");
4     sb.append(" * ");
5     sb.append(" FROM ");
6     sb.append(type.getSimpleName());
7     sb.append(" WHERE " + field + " =? ");
8     return sb.toString();
9 }
10
11 public T findById(int id) {
12     Connection connection = null;
13     PreparedStatement statement = null;
14     String query = createSelectQuery("id");
15     try {
16         connection = ConnectionFactory.getConnection();
17         statement = connection.prepareStatement(query);
18         statement.setInt(1, id);
19         resultSet = statement.executeQuery();
20
21         return createObjects(resultSet).get(0);
22     } catch (SQLException e) {
23         LOGGER.log(Level.WARNING, type.getName() +
24             "DAO:findById " + e.getMessage());
25     } finally {
26         ConnectionFactory.close(resultSet);
27         ConnectionFactory.close(statement);
28         ConnectionFactory.close(connection);
29     }
30     return null;
31 }
```

Take a look on this implementation:

https://gitlab.com/utcn_dsrl/pt-reflection-example

Reflection Techniques Example #4

- **Problem:** Propose a smart solution for constructing the DAO classes using Generics and Reflection - findByld method (cntd.)
- Line 1 - Given a result set, obtain the list of model object of type T
- For each (Line 5) *result* from ResultSet
 - ▶ Line 6 - Create a new instance of type T
 - ▶ Line 7 - For each *field* of the class T
 - ★ Line 8 - Retrieve from the current *result* the value of the current field
 - ★ Line 9-10 - Obtain the method for setting a value to the *field*.
 - ★ Line 11 - Using the obtained method set the value to the field

```
1 private List<T> createObjects(ResultSet resultSet) {  
2     List<T> list = new ArrayList<T>();  
3  
4     try {  
5         while(resultSet.next()) {  
6             T instance = type.newInstance();  
7             for (Field field: type.getDeclaredFields()) {  
8                 Object value = resultSet.getObject(field.  
9                     getName());  
10                PropertyDescriptor propertyDescriptor = new  
11                    PropertyDescriptor(field.getName(),  
12                        type);  
13                Method method = propertyDescriptor.  
14                    getWriteMethod();  
15                method.invoke(instance, value);  
16            }  
17            list.add(instance);  
18        }  
19    } catch (InstantiationException e) {  
20        // rest of the code  
21    }  
22 }
```

Take a look on this implementation:

https://gitlab.com/utc_n_dsrl/pt-reflection-example

Reflection Techniques Example #5

- **Problem:** Propose a smart solution for constructing the DAO classes using Generics and Reflection - Usage
- Line 1 - Define a class *StudentDAO* that extends the *AbstractDAO* and specify the model class used: *Student*
- The generic methods are directly accessed through inheritance
- Other specific methods can be implemented at this level

```
1  public class StudentDAO extends AbstractDAO<Student> {  
2      // use basic CRUD methods from superclass  
3      // TODO: create only specific queries  
4  }
```

Take a look on this implementation:

https://gitlab.com/utcn_dsrl/pt-reflection-example

Source code

- Download the source code from:
 - ▶ Simple layered project: https://gitlab.com/utcn_dsrl/pt-layered-architecture
 - ▶ Reflection example: https://gitlab.com/utcn_dsrl/pt-reflection-example